

P4044R0: Just `pre!`. Mandatory precondition for contracts

Document number: P4044R0

Date: 2026-03-08

Authors: Lucian Radu Teodorescu <lucteo@lucteo.ro>

Audience: EWG

Abstract

C++26 contracts support a wide range of uses, but they currently cannot reliably enforce *UB-safety*. Because preconditions may be evaluated with *ignore* semantics, a library that relies on contract preconditions to prevent undefined behavior cannot guarantee that those checks will execute.

Several proposals attempted to address this limitation (P3911R2 (<https://wg21.link/P3911R2>), P3919R0 (<https://wg21.link/P3919R0>), P4005R0 (<https://wg21.link/P4005R0>), P4009R0 (<https://wg21.link/P4009R0>)), but none achieved consensus. Concerns were raised that even small changes to the contracts facility might have unintended consequences.

This paper proposes a minimal mechanism to address RO 2-056. It restores an expected use of preconditions: terminating execution when a fundamental safety requirement is violated. The proposal stays entirely within semantics already permitted by the current draft.

The proposal introduces the syntax `pre!` to mark *mandatory preconditions*. Such preconditions are guaranteed to use *terminating semantics*. No new evaluation mode is introduced; the proposal only provides a way to require behavior that conforming implementations may already provide.

This proposal aims to resolve the RO 2-056 NB comment.

1. Introduction

1.1. Motivation and approach

In Kona 2025, EWG encouraged work (<https://github.com/cplusplus/nbballot/issues/631#issuecomment-3488277609>) in the direction proposed in D3911R0 (published as P3911R0 (<https://wg21.link/P3911R0>)). A revised version of the work, P3911R2 (<https://wg21.link/P3911R2>) was rejected by EWG in a telecon. The underlying RO 2-056 NB-comment was left unresolved. Follow-up attempts to resolve this NB still failed (P4005R0 (<https://wg21.link/P4005R0>), P4009R0 (<https://wg21.link/P4009R0>)).

P3911R2 (<https://wg21.link/P3911R2>) proposed an "always-contract-terminate" syntax and semantics for preconditions, postconditions and contract assertion statements. Some of the technical objections were concerned with the feasibility of implementing always-contract-terminate postconditions.

However, addressing the core of RO 2-056 only requires "always-contract-terminate" preconditions. This paper proposes the minimal mechanism needed to achieve that while remaining entirely within semantics already permitted by the current draft.

1.2. Contracts and UB-safety

In the seminal paper *Proving the Correctness of Multiprocess Programs* (<https://lamport.azurewebsites.net/pubs/proving.pdf>), Leslie Lamport describes program correctness in terms of *safety* and *liveness* properties. A *safety* property states that something bad does not happen, while a *liveness* property states that something good eventually happens. For example, "the program must not have undefined behavior" is a safety property, while "the program eventually terminates" is a liveness property.

Lamport reasons about such properties using *assertions* attached to program points. This resembles the role of assertions in the C++26 contracts facility. If the assertions corresponding to a set of safety properties always hold, the program is safe with respect to those properties.

Among safety properties, the absence of undefined behavior is fundamental. Once undefined behavior occurs, no other property can be guaranteed. We say that a program is *UB-safe* if it never exhibits undefined behavior.

In practice, UB-safety is often expressed through preconditions. If the preconditions of all functions hold, the program does not trigger undefined behavior.

However, C++26 contracts do not guarantee that preconditions will be evaluated. A program may configure contract evaluation to use *ignore* semantics. In that case, a library that encodes UB-safety requirements as preconditions cannot rely on those checks being executed.

To use contracts to enforce UB-safety, precondition violations must terminate execution. If contracts cannot guarantee terminating enforcement for such safety preconditions, it is unclear what mechanism library authors should use to express and enforce these interface requirements.

This proposal stays entirely within semantics already permitted by the current draft. It only provides a way to require terminating enforcement for specific preconditions.

2. Proposal

This paper proposes the following:

- Semantics: allow certain precondition contract assertions to always use *terminating semantics*.
- Syntax: introduce a new syntax (`pre!`) to require terminating semantics for a precondition.

In this paper, the term *mandatory precondition* refers to a precondition contract assertion introduced with the `pre!` syntax and therefore guaranteed to use *terminating semantics*.

2.1 Semantics

The current draft [basic.contract.eval#2] (<https://eel.is/c++draft/basic.contract.eval#2>) specifies that the evaluation semantics of a contract assertion are implementation-defined. Therefore, using *terminating semantics* for contract assertions is already permitted.

Moreover, the recommended practice ([basic.contract.eval#3] (<https://eel.is/c++draft/basic.contract.eval#3>)) states that, by default, contract assertions should use *enforce* semantics.

Requiring certain precondition contract assertions to use *terminating semantics* therefore introduces no new semantic behavior. An implementation that evaluates all preconditions with *terminating semantics* is already conforming. Programs written assuming terminating enforcement of preconditions are therefore compatible with conforming implementations.

This proposal merely provides a way for authors to require that already-permitted behavior for specific preconditions. The language therefore gains no new evaluation mode — only a mechanism to require an existing one.

2.2 Syntax

To distinguish mandatory preconditions from regular preconditions (which may use *ignore* or *observe* semantics), new syntax is required. While bikeshedding syntax is fun, we will restrain ourselves and just propose the `pre!` syntax, as described in P3911R2 (<https://wg21.link/P3911R2>) — P3911R2 provides enough motivation in support of `pre!`.

The syntax clearly signals that the precondition is mandatory. Alternative syntax may be considered if the committee prefers a different spelling.

3. Analysis

Advantages:

- **Minimal language change.** The proposal introduces a single syntactic form (`pre!`) and does not add any new evaluation semantics.
- **Relies on already-permitted behavior.** The current draft allows implementations to evaluate all preconditions with terminating semantics. An implementation that does so is already conforming.
- **No impact on existing contract assertions.** Existing `pre` contract assertions are unchanged. The new syntax is purely opt-in.
- **Supports library safety guarantees.** Library interfaces often express safety requirements through preconditions. Mandatory preconditions allow such requirements to terminate execution when violated, preventing undefined behavior.
- **Enables reliable enforcement of UB-safety requirements.** Preconditions that encode safety constraints can be guaranteed to terminate execution when violated.

- **Consistent with recommended practice.** The current draft recommends that contract assertions use *enforce* semantics by default. Mandatory preconditions make this behavior explicit where required.

Potential concerns:

- **Additional syntax.** The proposal introduces one new syntactic form. Alternative spellings could be considered if the committee prefers a different notation.

Overall, the proposal introduces a minimal syntactic mechanism to require behavior that is already permitted by the current contracts design, allowing the C++26 contracts facility to be used to improve program UB-safety.

4. Wording

[Editor's note: Apply the following change in [basic.contract.eval]:]

2. It is implementation-defined which evaluation semantic is used for any given evaluation of a contract assertion. If a contract assertion is introduced using the `pre!` syntax [decl.contract.func], the implementation shall evaluate the assertion using terminating semantics.

[*Note 1:* The range and flexibility of available choices of evaluation semantics depends on the implementation and need not allow all four evaluation semantics as possibilities. The evaluation semantics can differ for different evaluations of the same contract assertion, including evaluations during constant evaluation. — *end note*]

3. *Recommended practice:* An implementation should provide the option to translate a program such that all evaluations of contract assertions that are not introduced using the `pre!` syntax [decl.contract.func] use the ignore semantic as well as the option to translate a program such that all evaluations of contract assertions use the enforce semantic. By default, evaluations of contract assertions should use the enforce semantic.

[Editor's note: Apply the following change in [decl.contract.func]:]

function-contract-specifier:
 precondition-specifier
 postcondition-specifier

precondition-specifier:

```
pre attribute-specifier-seqopt ( conditional-expression )  
pre ! attribute-specifier-seqopt ( conditional-expression )
```

postcondition-specifier:

```
post attribute-specifier-seqopt ( result-name-introduceropt conditional-expression )
```

5. Suggested polls

5.1 Rationale

The discussion around this proposal can be framed through the following questions (for which the author believes the answer should be yes):

1. Should C++ aim to improve UB-safety?
2. Can contracts be used to help enforce UB-safety?
3. Is requiring terminating semantics for certain preconditions consistent with the C++26 contracts facility (no negative consequences)?
4. Is the `pre!` syntax acceptable for C++26?

Questions (1) and (2) establish the motivation for this proposal. If the committee answers *yes* to both, the key technical question becomes (3).

If the answer to (3) is *yes*, the remaining discussion concerns syntax, and the committee can proceed to consider the `pre!` spelling or possible alternatives.

If the answer to (3) is *no*, the consequences should be clearly identified. This proposal intentionally relies only on behavior already permitted by the current draft. If even such a minimal extension cannot be supported, it would indicate that the current contracts facility cannot support this expected use of preconditions.

5.2 Wording for suggested polls

Poll 1: Is requiring terminating semantics for certain preconditions consistent with the C++26 contracts facility (no negative consequences)?

Poll 2: Is the ``pre!`` syntax acceptable for C++26?

